LangChain

LangChain Nedir?

LangChain, büyük dil modelleri (LLM'ler) ile daha karmaşık ve çok adımlı uygulamalar geliştirmeyi sağlayan bir framework'tür.

LangChain Felsefesi?

LangChain, büyük dil modelleri (LLM'ler) ile uygulama geliştirmeyi kolaylaştırmayı hedefleyen, aynı zamanda esnek ve üretime uygun çözümler sunan bir framework'tür.

LangChain'in temel yaklaşımı şu prensiplere dayanır:

- Büyük dil modelleri güçlü ve dönüştürücü bir teknolojidir.
- LLM'ler, harici veri kaynaklarıyla birlikte kullanıldığında çok daha etkili hale gelir.
- LLM'ler, gelecekteki yazılım uygulamalarının yapısını değiştirecektir.
- Bu teknolojinin gelişimi henüz erken aşamadadır.

LangChain

Agents

Ajanlar, görevler üzerinde akıl yürütebilen, hangi araçların kullanılacağına karar verebilen ve çözüme doğru yinelemeli biçimde ilerleyebilen sistemler oluşturmak için dil modellerini araçlarla birleştirir.

Model

Model, ajanın akıl yürütme motorudur. Statik ya da dinamik şekilde belirtilebilir.

Statik Model

Statik modeller, ajan oluşturulurken bir kez yapılandırılır ve yürütme boyunca değişmez. En yaygın ve en yalın yaklaşım budur.

```
from langchain.agents import create_agent

agent = create_agent("openai:gpt-5", tools=tools)
```

Agents

Dinamik model

Dinamik modeller, mevcut durum ve bağlama göre çalışma zamanında seçilir. Bu, daha gelişmiş yönlendirme mantıkları ve maliyet optimizasyonu sağlar.

```python
from langchain_openai import ChatOpenAI
from langchain.agents import create_agent
from langchain.agents.middleware import wrap_model_call, ModelRequest, ModelResponse

basic_model = ChatOpenAI(model="gpt-4.1-mini")
advanced_model = ChatOpenAI(model="gpt-4.1")

@wrap_model_call
def dynamic_model_selection(request: ModelRequest, handler) -> ModelResponse:
    message_count = len(request.state["messages"])

    if message_count > 10:
        model = advanced_model
    else:
        model = basic_model

    return handler(request.override(model=model))

agent = create_agent(
    model=basic_model,
    tools=tools,
    middleware=[dynamic_model_selection]
)
```

Models

- LLM'ler, insan benzeri metni yorumlayıp üretebilen güçlü yapay zeka araçlarıdır. İçerik yazma, çeviri, özetleme ve soru cevaplama gibi görevleri her biri için ayrı özel eğitim gerekmeden gerçekleştirebilirler.
- Modeller, LangChain ajanlarının akıl yürütme motorudur. Hangi aracın çağrılacağına, araç sonuçlarının nasıl yorumlanacağına ve final yanıtın ne zaman verileceğine model karar verir.

Model başlatma

Bağımsız bir modeli başlatmanın en kolay yolu init_chat_model kullanmaktır:

```
from langchain.chat_models import init_chat_model

model = init_chat_model("gpt-5.2")
response = model.invoke("Why do parrots talk?")
```

Messages

Mesajlar, LangChain içinde modeller için bağlamın temel birimidir. Bir LLM ile etkileşimde konuşma durumunu temsil etmek için gereken içerik ve meta veriyi taşırlar.

- Bir mesaj şu unsurları içerir:
- Role: mesaj türünü belirtir
- Content: asıl içerik
- Metadata: yanıt bilgileri, mesaj kimliği, token kullanımı gibi isteğe bağlı alanlar

LangChain, tüm sağlayıcılarla çalışan standart mesaj tipleri sunar.

Messages

SystemMessage

- Modelin rolünü, tonunu ve cevaplama ilkelerini belirlemek için kullanılır.

HumanMessage

- Kullanıcı girdisini temsil eder; metin, görsel, ses ve diğer çok modlu içerikleri taşıyabilir.

```python
from langchain.chat_models import init_chat_model
from langchain.messages import HumanMessage, AIMessage, SystemMessage

model = init_chat_model("gpt-5-nano")

messages = [
    SystemMessage("You are a helpful assistant."),
    HumanMessage("Hello, how are you?")
]
response = model.invoke(messages)
```

AI message: model çıktısını temsil eder

Tool message: araç çağrısı sonuçlarını taşır

Tools

Araçlar, agents tarafının gerçek dünyayla etkileşim kurmasını sağlar. Böylece ajanlar anlık veri çekebilir, kod çalıştırabilir, dış veritabanlarını sorgulayabilir ve dış sistemlerde eylem gerçekleştirebilir.

Normalde LLM:

- sadece metin üretir

Ama tools ile:

- veri çeker
- API çağırır
- kod çalıştırır
- işlem yapar

Tools

LLM = Beyin | Tools = Eller

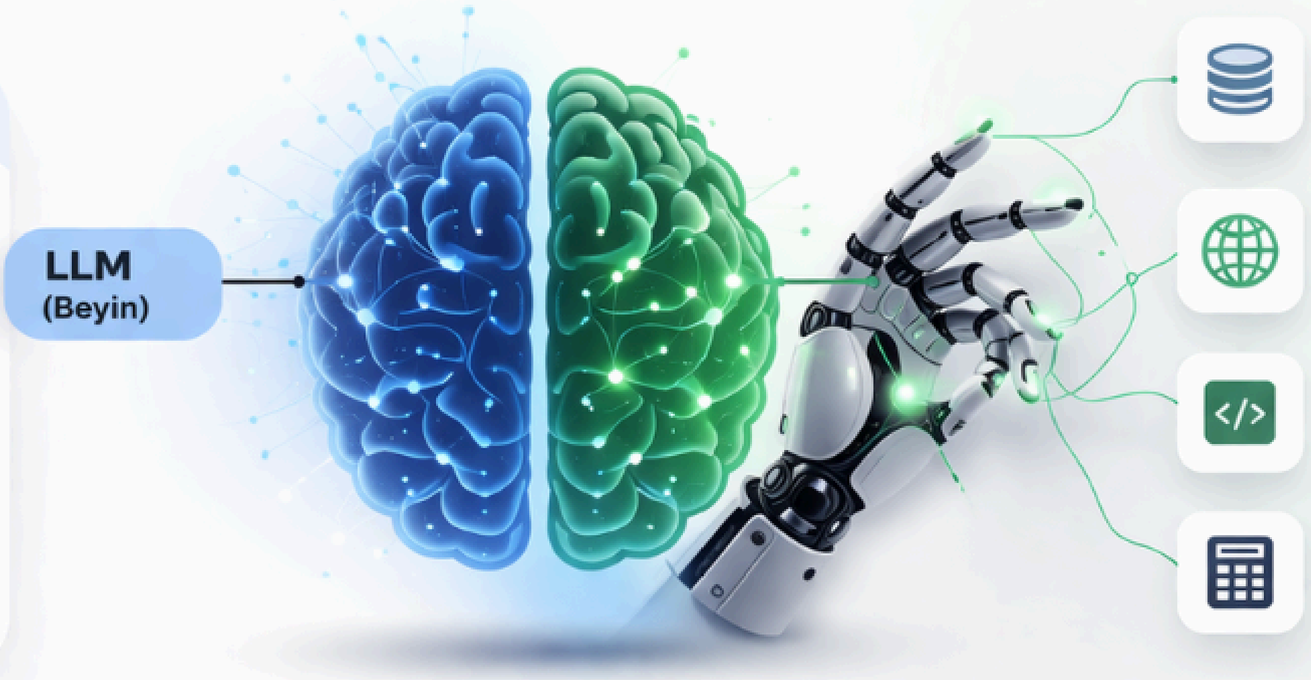
LLM (Beyin)

Anlama / Akıl Yürütme
Metni anlar, düşünür

Cevap Üretme
Sorulara yanıt verir

Karar Verme
Hangi adımı atacağını planlar

Bağlamı Hatırlama
Konuşma geçmişini tutar



Tools (Eller)

Veri Çekme
API / Veritabanı sorguları

Dış Sistemlerle Etkileşim
Web, API, Dosya işlemleri

Kod Çalıştırma
Python fonksiyonları

İşlem Yapma
Hesaplama / Analiz

Tools

En kolay yöntem @tool dekoratörüdür:

```python
from langchain.tools import tool

@tool
def search_database(query: str, limit: int = 10) -> str:
    """Search the customer database for records matching the query."""
    return f"Found {limit} results for '{query}'"
```

Araçlar, [ToolRuntime] üzerinden çalışma zamanı verilerine erişebilir:

- **State**: kısa süreli hafızaya
- **Context**: Sabit bilgilere
- **Store**: uzun süreli hafıza
- **Stream Writer**: uzun süren işlemlerde gerçek zamanlı güncelleme
- **Config**: çalışma yapılandırması

Short-term Memory

Kıse süreli bellek, önceki etkileşimler hakkında bilgi hatırlayan sistemdir. Ajanlar için bellek kritiktir; çünkü önceki etkileşimleri hatırlamalarına, geri bildirimlerden öğrenmelerine ve kullanıcı tercihlerine uyum sağlamalarına yardımcı olur.
Short-term memory, tek bir thread veya konuşma içindeki önceki etkileşimleri hatırlatır.
Uzun konuşmalar, LLM'ler için sorun yaratır:

- tam geçmiş bağlam penceresine sığmayabilir
- bağlam sığsa bile model eski veya alakasız içerikle dikkat dağıtabilir
- gecikme ve maliyet artabilir

Short-term Memory

Ajan düzeyinde kısa süreli bellek eklemek için create_agent içinde checkpointer verilmelidir:

```
from langchain.agents import create_agent
from langgraph.checkpoint.memory import InMemorySaver

agent = create_agent(
    "gpt-5",
    tools=[get_user_info],
    checkpointer=InMemorySaver(),
)
```

Structured Output

Yapılandırılmış Çıktı, yapay zekanın çıktısını serbest metin yerine belirli ve öngörülebilir bir formatta (JSON, Pydantic vb.) üretmesini sağlar.

Bu sayede çıktıyı sonradan ayrıştırmaya gerek kalmadan doğrudan uygulamalarda kullanabiliriz.

Structured output davranışı response_format parametresiyle kontrol edilir:

- Model:
  - Destekliyorsa → ProviderStrategy
  - Desteklemiyorsa → ToolStrategy
- LangChain uygun yöntemi otomatik seçer

Structured Output

```python
class ContactInfo(BaseModel):
    name: str
    email: str
    phone: str

agent = create_agent(
    model="gpt-5",
    response_format=ContactInfo
)
```

JSON

```json
{
  "name": "Enes",
  "email": "enes@gmail.com",
  "phone": "123"
}
```

Model çıktıyı hep bu formatta üretir.

Middleware

Middleware, LangChain ajanlarının çalışma sürecindeki farklı adımlara müdahale etmenizi sağlayan bir kontrol katmanıdır.
Model çağrıları, tool kullanımı ve veri akışı üzerinde değişiklik yapmanıza imkan verir.

- Hazır middleware'ler kullanılabilir
- İstenirse özel middleware yazılabilir
- Ajanın akışına araya giren katman gibi çalışır

Middleware

Ne İşe Yarar?

- Model davranışını çalışma anında değiştirme
- İstek ve yanıtları dönüştürme
- Loglama ve izleme ekleme
- Güvenlik ve politika kontrolleri uygulama
- Tool çağrılarından önce/sonra işlem yapma

Prebuilt Middleware

Bu middleware'ler üretime hazırdır ve yapılandırılabilir.

- Tool retry
- Model retry
- LLM tool emulator
- Context editing
- Shell tool
- File search
- Filesystem
- Subagent
- Summarization
- Human-in-the-loop
- Model call limit
- Tool call limit
- Model fallback
- PII detection
- To-do list
- LLM tool selector

Custom Middleware Nedir

Custom Middleware, ajan akışının belirli noktalarına özel müdahale eklemenizi sağlayan hook yapılarıdır.
Bu sayede sistem davranışı tamamen özelleştirilebilir.

1-) Node-style Hooks

Belirli adımlarda sırayla çalışır
before_agent, before_model
after_model, after_agent

Kullanım:
loglama
doğrulama
state güncelleme

```
@before_model(can_jump_to=["end"])
def check_message_limit(state: AgentState, runtime: Runtime) -> dict[str, Any] | None:
    if len(state["messages"]) >= 50:
        return {
            "messages": [AIMessage("Conversation limit reached.")],
            "jump_to": "end"
        }
    return None
```

Custom Middleware Nedir

2-) Wrap-style hooks

Model veya araç çağrılarının etrafında çalışır. Handler'ı:

- hiç çağırmayabilir
- bir kez çağırabilir
- birden çok kez çağırabilir

Kullanılabilir hook'lar:

- wrap_model_call
- wrap_tool_call

```python
@wrap_model_call
def retry_model(request: ModelRequest, handler) -> ModelResponse:
    for attempt in range(3):
        try:
            return handler(request)
        except Exception:
            if attempt == 2:
                raise
```

Frontend

LangChain'de frontend, ajanların ürettiği çıktıları kullanıcıya etkileşimli ve görsel şekilde sunan katmandır.
Basit mesaj gösteriminden, gelişmiş kontrol ve hata ayıklamaya kadar geniş bir yapı sunar.
Mimari

- Backend → createAgent (ajanı oluşturur)
- Frontend → useStream (veriyi çeker ve gösterir)

Frontend

Markdown Messages

LLM'ler (Büyük Dil Modelleri), ürettiği metinleri sadece düz yazı olarak değil, başlıklar, listeler, kod blokları, tablolar ve vurgulamalar gibi zengin biçimlendirmelerle Markdown formatında verir.

Eğer bu çıktıları sadece düz metin olarak gösterirsen, modelin oluşturduğu bu yapı ve görsel hiyerarşi kaybolur; yani başlıklar birbirine karışır, kod blokları ayrılamaz, tablolar düzgün görünmez.

Frontend

Tool Calling

Ajanlar hava durumu API'leri, hesap makineleri, web araması, veritabanı sorguları ve daha birçok harici aracı çağırabilir. Sonuçlar genellikle ham JSON olarak gelir. Bu desen, her araç çağrısı için yükleme ve hata durumlarıyla birlikte yapılandırılmış, tür güvenli UI kartları nasıl oluşturulacağını açıklar.

Frontend

Human-in-the-Loop

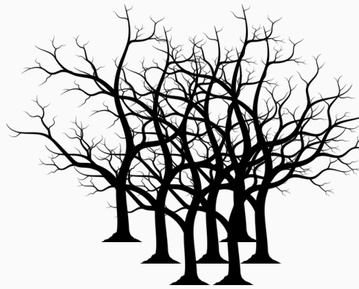
Her ajan eylemi gözetimsiz çalışmamalıdır. Ajan bir e-posta göndermeden, kayıt silmeden, finansal işlem gerçekleştirmeden veya geri döndürülemez bir operasyon yapmadan önce insan onayı gerekebilir. Human-in-the-Loop (HITL) deseni, ajanın duraklayıp bekleyen eylemi kullanıcıya göstermesine ve yalnızca onay geldikten sonra devam etmesine olanak tanır.

Frontend

Branching Chat

Yapay zeka ajanlarıyla yapılan konuşmalar çoğu zaman doğrusal değildir. Kullanıcı eski bir soruyu yeniden yazmak, beğenmediği bir yanıtı tekrar üretmek ya da önceki işi kaybetmeden bambaşka bir konuşma yolunu denemek isteyebilir. Branching chat, sohbet arayüzüne sürüm kontrolü mantığı getirir.

Bu modelde konuşma bir liste değil, bir ağaç olarak ele alınır. Her mesaj bir düğümdür. Bir mesajı düzenlemek veya bir AI yanıtını yeniden üretmek, o noktadan yeni bir dal oluşturur.



Guardrails

Guardrail'ler, ajan yürütmesinin kritik noktalarında içeriği doğrulayarak ve filtreleyerek güvenli, uyumlu yapay zeka uygulamaları geliştirmenize yardımcı olur. Hassas bilgileri algılayabilir, içerik politikalarını zorlayabilir, çıktıları doğrulayabilir ve sorunlar oluşmadan önce güvensiz davranışları engelleyebilir.

Guardrails

Yaygın kullanım alanları

- Kişisel verilerin sızmasını önlemek
- Prompt injection saldırılarını tespit edip engellemek
- Uygunsuz veya zararlı içeriği bloke etmek
- İş kuralları ile uyumluluk gereksinimlerini uygulamak
- Çıktı kalitesi ve doğruluğunu doğrulamak

Guardrails

Deterministik guardrail'ler

Regex, anahtar sözcük eşleştirme veya açık kontroller gibi kural tabanlı mantık kullanır. Hızlı, öngörülebilir ve maliyet açısından avantajlıdır; ancak nüanslı ihlalleri kaçırabilir.

Model tabanlı guardrail'ler

İçeriği anlamsal olarak değerlendirmek için LLM ya da sınıflandırıcı kullanır. Kural tabanlı yaklaşımın kaçırdığı daha ince sorunları yakalayabilir; fakat daha yavaş ve daha pahalıdır.

LangChain, hem yerleşik guardrail'ler hem de kendi güvenlik katmanlarınızı yazmanız için esnek middleware sistemi sunar.

Guardrails

Yerleşik guardrail'ler

LangChain, konuşmalardaki Kişisel Olarak Tanımlanabilir Bilgileri (PII) algılamak ve işlemek için yerleşik middleware sağlar. E-posta, kredi kartı, IP adresi ve benzeri veri türleri tespit edilebilir.

Desteklenen işleme stratejileri:

- redact: değeri [REDACTED_{PII_TYPE}] ile değiştirir
- mask: değerin bir kısmını gizler
- hash: deterministik bir hash ile değiştirir
- block: algılandığında hata fırlatır

Guardrails

```python
from langchain.agents import create_agent
from langchain.agents.middleware import PIIMiddleware

agent = create_agent(
    model="gpt-4.1",
    tools=[customer_service_tool, email_tool],
    middleware=[
        PIIMiddleware("email", strategy="redact", apply_to_input=True),
        PIIMiddleware("credit_card", strategy="mask", apply_to_input=True),
        PIIMiddleware(
            "api_key",
            detector=r"sk-[a-zA-Z0-9]{32}",
            strategy="block",
            apply_to_input=True,
        ),
    ],
)
```

Guardrails

İnsan sürece dahil guardrail'i

Yüksek riskli araçların çalıştırılmasından önce insan onayı istemek için yerleşik middleware bulunur. Finansal işlemler, dış taraflara e-posta gönderme veya üretim verisi silme gibi hassas operasyonlarda etkilidir.

```
agent = create_agent(
    model="gpt-4.1",
    tools=[search_tool, send_email_tool, delete_database_tool],
    middleware=[
        HumanInTheLoopMiddleware(
            interrupt_on={
                "send_email": True,
                "delete_database": True,
                "search": False,
            }
        ),
```

Model Context Protocol (MCP)

LLM ajanlarının araçlara ve bağlama erişimini standartlaştıran açık bir protokoldür. LangChain, langchain-mcp-adapters kütüphanesi sayesinde MCP sunucularındaki araçları kullanmayı sağlar ve ajanların güvenli, tutarlı ve yapılandırılmış bir şekilde harici sistemlerle etkileşim kurmasına imkan tanır.

Bu sayede ajanlar, karmaşık görevleri daha güvenilir ve tutarlı şekilde yerine getirebilir.

Human-in-the-loop

Human-in-the-Loop (HITL) middleware'i, ajan araç çağrılarına insan gözetimi eklemenizi sağlar. Model gözden geçirilmesi gereken bir eylem önerdiğinde yürütme durdurulabilir, karar beklenebilir ve daha sonra devam ettirilebilir.

Bu mekanizma, araç çağrılarını yapılandırılmış bir politika ile kontrol eder. Müdahale gerekiyorsa bir interrupt üretilir, yürütme durur ve durum LangGraph persistence ile saklanır.

İnsan kararı üç biçimde olabilir:

- approve: eylemi olduğu gibi onayla
- edit: eylemi değiştirerek çalıştır
- reject: eylemi reddet ve geri bildirim ver

Human-in-the-loop

```python
from langchain.agents import create_agent
from langchain.agents.middleware import HumanInTheLoopMiddleware
from langgraph.checkpoint.memory import InMemorySaver

agent = create_agent(
    model="gpt-4.1",
    tools=[write_file_tool, execute_sql_tool, read_data_tool],
    middleware=[
        HumanInTheLoopMiddleware(
            interrupt_on={
                "write_file": True,
                "execute_sql": {"allowed_decisions": ["approve", "reject"]},
                "read_data": False,
            },
            description_prefix="Tool execution pending approval",
        ),
    ],
    checkpointer=InMemorySaver(),
)
```

Multi-agent

Çok ajanlı sistemler, karmaşık iş akışlarını çözmek için özelleşmiş bileşenleri koordine eder.

Geliştiriciler genellikle şu ihtiyaçlar nedeniyle multi-agent yaklaşımı arar:

- Bağlam yönetimi: Modelin bağlam penceresini aşmadan uzman bilgiyi sunmak
- Dağıtık geliştirme: Farklı ekiplerin yetenekleri bağımsız geliştirebilmesi
- Paralelleştirme: Alt görevler için uzman işçiler üretip eşzamanlı çalıştırmak

Multi-agent desenleri özellikle şu durumlarda değerlidir:

- Tek bir ajan çok fazla tool içerdiğinde
- Uzun istemler ve alan özel araçlar gerektiğinde
- Belirli koşullar sağlanmadan bazı yeteneklerin açılmaması gerektiğinde

Retrieval

Büyük Dil Modelleri güçlüdür; ancak iki temel sınıra sahiptir:

- Sonlu bağlam penceresi vardır, yani tüm bilgi yığınını tek seferde alamazlar.
- Bilgileri statiktir; eğitim verileri belirli bir zamanda donar.

Retrieval, sorgu anında ilgili harici bilgiyi getirerek bu sorunları çözer. Bu, Retrieval-Augmented Generation (RAG) yaklaşımının temelidir.

Retrieval

Retrieval, LLM'lerin çalışma anında ilgili bağlama erişmesini sağlar. Gerçek dünyadaki uygulamalar ise genellikle retrieval'i üretimle birleştirerek bağlama dayalı yanıt üretir. Bu yaklaşımın adı RAG'dir.

LangChain dokümanı üç ana yaklaşımı açıklar:

1-) 2-Step RAG

Retrieval her zaman üretimden önce yapılır. Basit, öngörülebilir ve hızlıdır. Dokümantasyon botları ve SSS sistemleri için uygundur.

Retrieval

2-) Agentic RAG

LLM destekli ajan, bilgiye ne zaman ve nasıl erişeceğine çalışma anında karar verir. Esnektir fakat gecikmesi değişkendir.

3-) Hybrid RAG

2-Step ve Agentic RAG özelliklerini birleştirir. Sorgu iyileştirme, retrieval doğrulama ve yanıt kontrolü gibi ara adımlar içerebilir.

Uzun Süreli Bellek

LangGraph, uzun süreli anıları bir store içinde JSON belgeleri olarak saklar. Her kayıt:

- özel bir namespace altında gruplanır
- benzersiz bir key ile tutulur

Bu yapı, kullanıcı kimliği, organizasyon kimliği veya başka etiketlerle hiyerarşik düzenleme yapmayı kolaylaştırır. İçerik filtreleriyle namespace'ler arası arama da desteklenir.

Uzun Süreli Bellek

```python
from langgraph.store.memory import InMemoryStore

def embed(texts: list[str]) -> list[list[float]]:
    return [[1.0, 2.0] * len(texts)]

store = InMemoryStore(index={"embed": embed, "dims": 2})
user_id = "my-user"
application_context = "chitchat"
namespace = (user_id, application_context)
store.put(
    namespace,
    "a-memory",
    {
        "rules": [
            "User likes short, direct language",
            "User only speaks English & python",
        ],
        "my-key": "my-value",
    },
)
```

Test

Ajanik uygulamalar, LLM'in kendi sonraki adımlarını seçmesine izin vererek yüksek esneklik ve özerklik sağlar. Ancak modelin kara kutu yapısı nedeniyle, ajan içindeki bir değişikliğin tüm sistemi nasıl etkileyeceğini önceden tahmin etmek zordur. Bu yüzden üretime hazır ajanlar için kapsamlı testler şarttır.

Önerilen üç temel test yaklaşımı:

- Unit tests
- Integration tests
- Evals

Test

(Integration Testing)

- Integration test'ler, ajanın model API'leri ve harici servislerle birlikte doğru çalıştığını doğrulamak için yapılır. Unit test'lerden farklı olarak gerçek ağ çağrıları yapılır; böylece bileşenlerin birlikte çalışması, kimlik bilgileri, şema uyumu ve gecikme ölçülebilir.
- Ajanın gerçek dünyadaki koşullarda güvenilir çalıştığını doğrulamak için kritik öneme sahiptir

```
import pytest

@pytest.mark.integration
def test_agent_with_real_model():
    agent = create_agent("claude-sonnet-4-6", tools=[get_weather])
```

Test

(Integration Testing)

- Ajanın ne kadar iyi performans gösterdiğini yürütme izini değerlendirerek ölçer. Integration test'ler temel doğruluğu kontrol ederken evals; prompt, tool veya model değişikliklerinden sonra kalite düşüşlerini yakalamakta kullanılır.
- Bir evaluator, ajan çıktısını ve gerekirse referans çıktıyı alıp bir skor döndürür:

```
def evaluator(*, outputs: dict, reference_outputs: dict):
    output_messages = outputs["messages"]
    reference_messages = reference_outputs["messages"]
    score = compare_messages(output_messages, reference_messages)
    return {"key": "evaluator_score", "score": score}
```

LangSmith Studio

LangChain ile ajanları yerelde geliştirirken ajanın içinde neler olduğunu görselleştirmek, gerçek zamanlı etkileşim kurmak ve sorunları ayıklamak faydalıdır. LangSmith Studio, yerel makinenizde çalışan LangChain ajanlarını geliştirmek ve test etmek için ücretsiz bir görsel arayüz sunar.

Studio, yerelde çalışan ajana bağlanır ve şu adımları gösterir:

- modele gönderilen prompt'lar
- araç çağrıları ve sonuçları
- ara durumlar
- final çıktı

Teşekkürler